Before the Copilots: Reanimating Simon's Heuristic Compiler (1961) and MIT's KBEmacs (1984)

Jeff Shrager, *Bennu Climate, Inc., Palo Alto, CA 94301 USA*

Abstract—AI-assisted programming is often presented as a new phenomenon, but the idea that a machine might write or help write programs is as old as artificial intelligence itself. We describe the reanimation of two historical AI programming systems from their original source listings. The first is Herbert Simon's Heuristic Compiler (1961), an IPL-V program that treated writing a program as a problem to be solved by the means-end analysis of the General Problem Solver. The second is KBEmacs (1984), the Knowledge-Based Editor of MIT's Programmer's Apprentice project, which let a programmer build programs by combining standard algorithmic "clichés" through a stylized English command language. Both run again: the Heuristic Compiler reproduces the compiled code printed in Simon's 1963 paper, and KBEmacs reproduces one of its own December 1984 self-tests exactly. Together they illustrate two historical approaches—autonomous problem solving and the knowledgeable apprentice—whose concerns reappear, in different technical form, in today's AI coding assistants.

Index Terms: *artificial intelligence history, automatic programming, AI-assisted programming, digital preservation, Heuristic Compiler, IPL-V (Information Processing Language V), KBEmacs, Lisp Machine, Programmer's Apprentice, H. A. Simon, software archaeology, R. C. Waters*

INTRODUCTION

Programmers in the 2020s increasingly work alongside AI systems that complete, write, explain, test, and repair code. These systems, built on large language models trained on enormous corpora of source code [3], [6], are widely described as something new under the sun. In one sense they are: nothing like their scale, fluency, or breadth existed before. But the *aspiration* they embody, a machine that takes part in the work of programming, is nearly as old as programming. In the 1950s "automatic programming" meant what we now call compiling: FORTRAN was introduced as "the FORTRAN automatic coding system" [1]. Within a few years of the founding of artificial intelligence, researchers were asking whether the methods of AI could be turned on the programming task itself [2], [17].

This paper describes the reanimation, from their original source listings, of two historical systems that represent two quite different answers to that question.

The first is Herbert A. Simon's *Heuristic Compiler*, written in IPL-V in 1961 and described in a RAND report that year [23] and in the *Journal of the ACM* in 1963 [25]. Simon asked "whether the task of writing a computer program can be regarded as a 'problem' in the sense in which that term is used in GPS," the General Problem Solver of Newell, Shaw, and Simon [10]. The Heuristic Compiler is, in effect, GPS turned on programming: given a description of what a routine should do, it searches for a program by finding differences between what it has and what it wants, and applying operators relevant to those differences.

The second is *KBEmacs*, the Knowledge-Based Editor in Emacs, built by Richard C. Waters with

XXXX-XXX © YYYY IEEE
Digital Object Identifier 10.1109/XXX.0000.0000000

Kent Pitman, Charles Rich, and others at the MIT Artificial Intelligence Laboratory as the culminating demonstration system of the Programmer's Apprentice project [28], [30], [16]. KBEmacs did not try to write programs on its own. It was an *assistant*: the programmer chose which standard algorithms ("clichés") to combine, and told KBEmacs so in a stylized English command language; KBEmacs supplied the details and wrote, and continually rewrote, the code.

The two systems are separated by a quarter century, by programming language (IPL-V on 1960s mainframes versus Zetalisp on the Symbolics 3600 Lisp Machine), and by research tradition (Carnegie Tech and RAND versus MIT). Yet between them they stake out most of the conceptual territory that today's AI coding tools occupy: how a programming task is specified, where the knowledge needed to carry it out comes from, how work is divided between human and machine, and how the system checks and explains what it has done.

We approached both as exercises in what has been called *executable archaeology* [21], [9]: studying historical computational systems by reconstructing and running their original code, rather than reimplementing their algorithms. The Heuristic Compiler was transcribed card by card from a 1961 line-printer listing in Simon's papers and run on a new IPL-V interpreter; KBEmacs was transcribed from a four-volume January 1985 printout and run under a new Zetalisp compatibility layer for a modern Common Lisp. In both cases the reanimation surfaced details invisible in the published accounts. And, in a turn that would surely have amused both Simon and Waters, both reanimations were themselves carried out with substantial help from a modern AI coding assistant.

HISTORICAL BACKGROUND

From Automatic Coding to Heuristic Programming

The phrase "automatic programming" has meant different things in different decades, usually whatever was just beyond the current state of the art [2], [17]. For the designers of FORTRAN it meant translating algebraic formulas into machine code [1]. Such translators are deterministic: the programmer has already decided *how* the computation is to be done, and the compiler maps that decision into another notation.

Newell, Shaw, and Simon's work at RAND and Carnegie Tech in the 1950s took a different view of what computers could do. Their Logic Theorist (1956) searched for proofs of theorems in *Principia Mathematica* [11], [9], and their General Problem Solver (1959) generalized its methods into *means-end analysis* [10]. Simon's 1963 summary of the method is worth quoting in full, because the Heuristic Compiler implements it almost literally:

> *Given the present and desired objects, find a difference between them. Next, find an operator relevant to the difference; determine if the operator can be applied to the present object. If so, apply it. (If not, describe the objects to which it would apply and transform the present object into an object of that kind—a new "desired object.") Take the new object thus obtained as the present object and repeat the process. [25, p. 494]*

Simon then made the move that defines the Heuristic Compiler. A proof is a sequence of operators that transforms axioms into a theorem; read the other way, it is a program that generates the theorem from the axioms. "Thus a theorem-proving system can be regarded, at least formally, as a program-writing system. Conversely, if we can formulate a programming goal as a difference between a present and a desired object, we can presumably use the same processes ... to generate a program" [25, p. 495]. The same observation, in a deductive rather than heuristic setting, later became the foundation of program synthesis from theorem proving [5], [8].

Simon described the program in RAND report P-2349 (June 30, 1961), which included "a listing of the program as of June 1961" [23]; in the *Journal of the ACM* in 1963 [25]; and at greater length in RAND Memorandum RM-3588-PR, *The Heuristic Compiler* (May 1963), whose appendices list a 1963 version of the compiler and a small companion program for "annexing" information stated as definite descriptions [26]. He was explicit about its status: "no claims will be made for the heuristic programming procedures described here as practical approaches to the construction of compilers. Their interest lies in what they teach us about the nature of the programming task" [25, p. 493].

The Programmer's Apprentice

The Programmer's Apprentice (PA) project began at MIT in the mid-1970s, with Charles Rich and Howard Shrobe's *Initial Report on a LISP Programmer's Apprentice* [15]. Its premise was that fully automatic pro-

gramming of realistic software was a distant goal, and that a more productive near-term aim was an intelligent *assistant* that would share with an expert programmer a body of knowledge about programs. Waters later put it this way: “Recognizing that it will be a long time before it is possible to fully duplicate an expert programmer’s abilities, the project seeks to develop an intelligent assistant system ... which will help a programmer in various phases of the programming task” [30, p. 47].

Over the following decade the project developed three central ideas. *Clichés* are standard computational methods, such as a linear search, a report generator, or successive approximation, each with *roles* that vary from use to use and a fixed surrounding structure. *Plans* are a language-independent representation of programs as hierarchical flowcharts with explicit data and control flow, which Rich formalized as the Plan Calculus [16]. And the *assistant approach* itself: a division of labor in which the programmer makes the design decisions and the apprentice handles the routine details. Waters’s dissertation developed the analysis of programs into plans [27].

KBEmacs was the realization of these ideas as a working editor. It ran on Symbolics Lisp Machines inside the Zmacs editor, and handled both Lisp and Ada. Its best-known demonstration builds an Ada report generator, `UNIT-REPAIR-REPORT`, from five commands such as “Define a simple-report procedure UNIT-REPAIR-REPORT” and “Fill the enumerator with a chain-enumeration of UNITS and REPAIRS” [30, p. 48],[29]. Waters pointed to the order-of-magnitude difference between the size of the command sequence and the size of the resulting program as a measure of the leverage KBEmacs provided, while noting that program construction is only a small fraction of the software life cycle [30, p. 48].

KBEmacs was candidly a research prototype. By 1986 Waters reported that “knowledge-based operations on large programs can take longer than 5 minutes,” that the system handled “only about 50% of Ada” and 90% of Lisp, that it knew “only a few dozen clichés” where a practical tool “would need to know many hundreds,” and that, its ideas having been used “for essentially all [they are] worth,” KBEmacs “has been mothballed” [30, pp. 54–55]. The project continued with work on the Plan Calculus, on automated reasoning, and on requirements and design apprentices, culminating in Rich and Waters’s 1990 book [18].

REANIMATING THE HEURISTIC COMPILER

The Listing

The surviving source is a line-printer listing headed `HEURISTIC CODER 7/16/61`, preserved in the Herbert A. Simon Papers at the Carnegie Mellon University Archives [24]. It was located by Mia Golec and Emily Davis of CMU with the author. Dated two weeks after RAND report P-2349, it is very nearly the program that report describes. The 95 scanned pages print the complete card deck, one card per line, with comments, IPL fields, and card identifiers. One page was scanned twice; the remaining 94 hold 4,473 cards, close to the pencilled total of “4464” on the first page. The listing also carries pencil: check marks, stack-depth counts beside a few routines, and a handful of proposed corrections, apparently the original debuggers’ notes.

The deck turned out to contain three things: an executive and two compilers; a natural-language front end (a character scanner, dictionary, and parser) intended to let the compiler accept English definitions; and a small “world” of knowledge about IPL-V, encoded as data, that the compilers reason about. The listing ends with the data and has no start card, so the program was evidently started by a separate card or deck.

Everything Is a Description List

The central representational idea of the Heuristic Compiler is that a *routine* is an object with attributes, stored as an IPL-V description list (a property list, in later Lisp terms). Each attribute is one possible definition of the routine: its IPL name, its *functional description* (a process name plus arguments, such as “INSERT ( ) AT END OF ( )”), its *state description* (which cells it affects, and what each holds before and after), its flow diagram, and its IPL-V code. Below that level, an IPL instruction is itself a description list with attributes for each field, and a symbol such as `J3` is a description list with a region (the letter J) and a location (the integer 3). The program never handles IPL code as text; it builds code out of these structures and prints it only at the end. Simon remarked that description lists gave IPL-V “expressive capabilities not readily available in more usual computer command languages” [25, p. 506].

The executive (routines U135–U138) is the paper’s general compiler, almost card for card. It finds the “closest” definition the routine has, looks up in a table of connections the compiler relevant to that kind of definition, applies it, and tests whether the routine has moved to a better kind of definition. If not, it gives up. It

TABLE 1. The two reanimated systems.

	Heuristic Compiler	KBEmacs
Principal author(s)	H. A. Simon (with H. S. Kelly)	R. C. Waters, K. Pitman, C. Rich, and others
Institution	Carnegie Tech / RAND	MIT AI Laboratory
Surviving source	Listing "HEURISTIC CODER 7/16/61"	Four-volume printout, January 1985 (KBE 358.0x)
Language / platform	IPL-V / (machine not established)	Zetalisp / Symbolics 3600, Release 4.5
Size	about 4,470 cards	87 files, about 34,500 lines transcribed
Input	state or functional description	stylized English commands
Central method	means-end analysis (GPS)	instantiating and combining clichés in plans
Reanimated on	new Common Lisp IPL-V interpreter	Zetalisp compatibility layer for SBCL
Benchmark reproduced	code printed in Simon (1963)	self-test of December 14, 1984

is a small means-end loop whose *differences* are kinds of definition and whose *operators* are whole compilers.

The *state description compiler* (U140–U169 and X5–X7) is the most GPS-like part. It compares the input and output states of each affected cell, classifies the biggest difference (addition, deletion, replacement, or copy), looks up an operator relevant to it, binds the operator's variables to the actual cells, and then recursively compiles two subproblems: getting from the original input state to the state the operator requires, and getting from the operator's output state to the desired output. Its knowledge of IPL-V consists of exactly two operators, LOAD and REPLACE, each itself a tiny routine with its own state description.

The *functional description compiler* (U134, U130, and helpers) works by analogy. Given a routine described as, say, "INSERT (1) AT END OF THE VALUE OF THE ATTRIBUTE (0) OF (2)," it searches the routines already compiled for one with the same *process*, here "INSERT (0) AT END OF (1)," whose code is the single primitive `J65`. It then adapts that routine to the new arguments: it moves inputs into working cells, compiles code to fetch each argument, recursing when an argument is itself a described computation (a "determiner"), and wraps the result in instructions that save and restore the working cells.

Running It

The deck runs on a Common Lisp IPL-V interpreter originally written to run the Logic Theorist [21], [22], extended to read 80-column card images directly and to supply J-functions the Heuristic Compiler uses but the Logic Theorist did not. Several existing J-functions turned out to be wrong in ways the Logic Theorist never exercised; after they were corrected against the 1964 manual [12], the interpreter was rechecked and still reproduces Stefferud's 1963 Logic Theorist output for all 24 of its theorems [19].

One print routine, U126, is called but absent from the 1961 listing; it was supplied from the 1963 listing in RM-3588-PR, Appendix A [26], and affects only printing. With that and a start card for the demonstration routine T1 (the 1963 listing ends with exactly such a card), the program runs Simon's published examples.

Compiling from a functional description. Given the functional description of INSERT AT END OF VALUE LIST, the program produces `J13 J52 11W2 11W0 J10 11W1 J65 J32 0`, character for character the code in the 1963 paper [25, pp. 499–500] (Figure 1). A print left in routine U113 shows the code growing one argument at a time, which happens to reproduce the sequence of stages through which the paper walks its readers. We note one qualification. In Simon's runs, as in ours, the functional description is supplied as a hand-coded list structure, not as typed English; the phrase is printed from it, but the compiler matches processes by symbol identity. The English front end in the deck has not yet been run (see "Next Steps").

Compiling from a state description. Simon's running example of the state description compiler is J3, SET SIGNAL MINUS: a routine that replaces the top symbol of the signal cell H5 with the symbol J3 ("minus"). The state description says that H5 goes from *some symbol on top of the rest of its pushdown list* to *J3 on top of the same rest*. The compiler finds a replacement difference in a named cell; finds REPLACE relevant, with its variable bound to H5, giving `20H5`; sets up the prefix subproblem "get J3 into the communication cell," an addition, for which LOAD is relevant, giving `10J3`; finds no suffix subproblem; and assembles `10J3 20H5 0`, the code in the paper [25, p. 498].

There is a catch, and it is the kind of thing only running the program reveals. *As listed, the deck does not compile J3.* The listing's J3 has both a functional and a state description, and the executive always prefers the functional description, on the stated assumption that "it is easier to compile from a functional description than from a state description" [25, p. 502].

```
FIND THE VALUE-OF THE ATTRIBUTE (0) OF (1)              <- an already-compiled routine

                                    J10   0

                                    J65   0             <- the code of INSERT (0) AT END OF (1)

                                   11W1                 <- fetch argument (1), now in W1
                                    J65   0

                                   11W2                 <- fetch the determiner's arguments
                                   11W0                    and apply its code (J10)
                                    J10
                                   11W1
                                    J65   0

J13
INSERT (1) AT END OF  THE VALUE-OF THE ATTRIBUTE (0) OF (2)

                              J13   J52                 <- preserve working cells W0-W2
                                   11W2
                                   11W0
                                    J10
                                   11W1
                                    J65
                                    J32   0             <- restore them and exit
```

FIGURE 1. The 1961 Heuristic Compiler, running in 2026, compiling INSERT AT END OF VALUE LIST from its functional description. The intermediate stages are printed by the program itself (a print in routine U113 that appears to be a debugging aid left in the deck). The final code is character for character the code printed in Simon's 1963 paper. Annotations after <- are ours.

No already-compiled routine shares J3's process, the functional description compiler fails, the executive sees no progress, and it gives up without ever trying the state description. The paper's J3 result is obtained only when J3's functional description is removed before the run, after which the state description compiler produces exactly the published code. The 1963 listing in RM-3588-PR suggests what happened: its executive is card for card the same, but the cards that give J3 a functional description are numbered as later insertions. Most likely the paper's J3 came from a version in which J3 had only a state description. (The 1963 state description compiler is also simpler in its representation of states, holding bare symbols where the 1961 version holds described ones.)

Flow diagrams. The paper's last section handles routines with branches and loops by decomposing them into *segments* joined by a flow diagram. The program reassembles routine J77 ("test whether there is a symbol equal to 0ACCUMULATOR on list 1ACCUMULATOR") from its segments, and decomposes it again into the paper's flow diagram.

The information annexer. RM-3588-PR's Appendix B, a small program that stores statements such as "X114 is the X33 of the X25 of X105" into description lists, either identifying an object already in memory or creating one, was also transcribed (409 cards) and runs. Its output matches the fragment of 1963 output that survives in the memorandum [26], [19].

REANIMATING KBEMACS

The Listings

The surviving KBEmacs source is a January 1985 printout, four bound volumes of Zetalisp listings, preserved and scanned by Kent M. Pitman, a member of the KBEmacs team [14]. It covers the system as of KBE version 358.0x (December 1984), although individual files carry their own dates and versions. Volume 1 opens with something unusual and extremely useful to an archaeologist: the results of KBEmacs's automatic self-test, run "Friday the fourteenth of December, 1984; 2:25:23 pm" on the Lisp Machine *Merlin*. The self-test runs six command sequences, compares the code KBEmacs generates with expected code, and reports that three sequences (HASH, SQRT, and SUMLIST) succeeded and three (BUCKET, DELETE, and MEMQ) failed with "Bad code generated."

The 87 files were transcribed with the assistance of a large language model into about 34,500 lines of source [20], roughly consistent with Waters's description of the system as "some 40,000 lines of Lisp code" [30, p. 55]. The transcription was initially unchecked; it was checked chiefly by *executing* it.

Text and Plans

A KBEmacs user works in a stylized English command language phrased in terms of clichés and their roles. The cliché library is written as code with holes, as in the definition of successive approximation in Figure 2.

But clichés are not stored as text. When the library loads, each definition is run through KBEmacs's *analyzer*, which turns code into a plan: a set of facts in a small database describing segments, ports, data flow, control flow, and roles. In a plan, a `LOOP` with a `SETQ` disappears; what remains is a temporal composition of a sequence of states. The knowledge-based editor operates on plans, and after every command the *coder* turns the plan back into program text, deciding where to nest expressions and where to introduce variables, and preferring names the programmer has already used. Waters called this two-way translation the *representation shift*, and argued that it gave plans locality, additivity, and modularity that text lacks [30]. Because the user may also simply edit the text, KBEmacs must keep text and plan consistent in both directions.

Running It

KBEmacs depends heavily on its environment: Flavors (the Lisp Machine's object system), the Zmacs editor, Lisp Machine processes, Waters's LetS loop notation, and Zetalisp itself. The reanimation targeted the *headless* core (the database, analyzer, cliché library, command language, knowledge-based editor, coder, and explanation generator), leaving out the editor, windows, actors, and the Ada front end. It runs on SBCL, a modern Common Lisp, through a compatibility layer comprising an existing Common Lisp port of Spice Lisp Flavors extended with the Lisp Machine's `:case` method combination, a port of Waters's LetS [4], a Zetalisp reader, a library of Zetalisp functions and conditions, and stand-ins for editor-level facilities the core needs. A small driver replays a self-test sequence from the 1984 record and compares the generated code with the expected code, as the original test harness did.

The SQRT sequence (Figure 3) builds a square-root routine from three clichés in seven commands, and then builds a variant with an extra parameter in seven more. After the first command that names a cliché, KBEmacs holds the cliché's structure with its roles still open as holes; each further command fills a role with code or with another cliché, whose own unfilled roles appear as new holes. After seven commands the plan has 32 segments and the program is complete.

All fourteen commands run, and both generated programs are `equal` to the code recorded in 1984. The generated code also works: `(mysqrt 2.0)` returns 1.4142157.

Getting there was instructive. Three kinds of problem had to be solved, and their distribution says something about where the difficulty of reanimation lies.

Transcription errors. About two dozen typo-level corrections were needed, each checked against the scans and logged [20]. In the listing font, quote and backquote are nearly identical, and the commonest error was one read as the other. The most insidious were *compensating* parenthesis errors: a missing parenthesis in one place and an extra one a few lines later, so that the file still balanced. In one case in the coder, two such pairs silently nested about thirty function definitions inside earlier ones, so that they were never defined. In another, in the analyzer, a clause of a conditional migrated into the wrong branch, so that moving a segment into a loop silently did nothing and the data-flow simplifier retried the move forever. Neither is detectable by checking balance; both were found by running the code and then confirmed against the scans. In one case the transcription faithfully preserved an error in the copy-and-paste sense: a line in the verb-definition macro had been copied from the macro just above it, shifting every verb's arguments by one.

Semantics that survive only in the code. Several behaviors of Zetalisp in Release 4.5 had to be inferred from how KBEmacs depends on them, because the system will not run otherwise. On the Lisp Machine, `(listp nil)` is false. Characters behave as integers. The function `map` maps over successive tails of a list. Most strikingly, `(boole 4 x y)` does not mean what it means in Common Lisp: the code is a PDP-10 Boolean-instruction code, and 4 means "x and not y," which KBEmacs uses for set difference in its live-variable analysis. With the Common Lisp meaning, the data-flow fixed point never converges. KBEmacs's sets are kept sorted by machine address, relying, as a comment admits, on the garbage collector never running (the 1984 self-test header duly records "Garbage collector enabled: No"); addresses are emulated by creation order.

Bugs in the reanimation's own tools. One persistent failure turned out to be in the modern port of LetS, which dropped the value of its `done` form; KBEmacs

```
(DEFINE-CLICHE SUCCESSIVE-APPROXIMATION
   (PRIMARY-ROLES (INITIAL-APPROXIMATION)
    DESCRIBED-ROLES (INITIAL-APPROXIMATION CRITERION IMPROVEMENT-STEP)
    COMMENT "iteratively improves an approximation until {the criterion} is
             satisfied" ...)
  (LET* ((APPROXIMATION {the input initial-approximation}) )
    (LOOP DO
      (IF ({the criterion} APPROXIMATION) (RETURN APPROXIMATION) )
      (SETQ APPROXIMATION ({the improvement-step} APPROXIMATION) ))))
```

FIGURE 2. A cliché from KBEmacs's library (`kbelib.lisp`, 1984). The braces are KBEmacs read syntax marking *roles*, the parts that vary from use to use; the surrounding code is fixed.

```
>Reinitialize database.
>Define a program MYSQRT with a parameter NUM.
>Implement it as a successive-approximation.
>Implement the initial-approximation as [1.] .
>Implement the improvement-step as an average of [APPROXIMATION]
   and [(// NUM APPROXIMATION)] .
>Implement the criterion as an equality-within-epsilon of
   [(* APPROXIMATION APPROXIMATION)] and [NUM] .
>Implement the epsilon as [0.0001] .

(DEFUN MYSQRT (NUM)
  (LET* ((APPROXIMATION 1))
    (LOOP DO
      (IF (< (ABS (- (* APPROXIMATION APPROXIMATION) NUM)) 0.0001)
          (RETURN APPROXIMATION))
      (SETQ APPROXIMATION (// (+ APPROXIMATION (// NUM APPROXIMATION)) 2.0)))))

The program MYSQRT is a successive-approximation. The initial-approximation is 1.
The criterion is an equality-within-epsilon ...
```

FIGURE 3. KBEmacs, running in 2026, replaying the first half of the SQRT sequence from its December 14, 1984 self-test. The generated code is `equal` to the code recorded in 1984. (`//` is Zetalisp's division operator.) The final lines are KBEmacs's own description of the program, generated from its plan, as recorded in the 1984 self-test (abridged).

uses that value to choose fresh argument names, and without it plans acquired ports with no name.

THEN AND NOW: HISTORICAL APPROACHES TO AI-ASSISTED CODING

Neither the Heuristic Compiler nor KBEmacs is an ancestor of today's AI coding assistants in any direct technical sense. Large language models are trained, not programmed with explicit operators or clichés, and they operate on program text rather than on description lists or plans. Nonetheless the questions these two systems were built to explore are recognizably the questions that shape AI-assisted programming today. Table 2 summarizes the comparison; we discuss five themes.

How is a programming task specified? Simon's two compilers differ precisely in how the task is described. A state description says what the machine looks like before and after, which is close to what we would now call a specification by examples or by pre- and postconditions, and close to the role unit tests play in directing a modern assistant. A functional description is an imperative phrase, which is close to a natural-language prompt. Simon drew a conclusion that remains apt: programmers use both, but "at the higher levels ... a language of functions appears generally the more natural and powerful," and "the mappings from functional descriptions to process language are simpler and more direct than the mappings from state descriptions" [25, p. 506]. KBEmacs chose the functional side and gave it a vocabulary of named algorithms. Today's assistants accept both kinds of

TABLE 2. Three approaches to machine participation in programming.

	Heuristic Compiler (1961)	KBEmacs (1984)	LLM-based assistants (2020s)
Task specification	state or functional description	English commands naming clichés	natural language, code context, tests
Source of knowledge	hand-coded operators and routines	hand-built cliché library (dozens)	learned from large code corpora
Internal representation	description lists	plans (data and control flow)	program text (tokens)
Core method	means-end analysis	cliché instantiation, constraints	learned generation; agentic loops
Reuse	adapt a compiled routine with same process	instantiate a cliché	implicit, plus retrieval of context
Division of labor	autonomous	programmer decides, apprentice implements	ranges from completion to autonomy
Checking	progress test on kind of definition	self-test; consistency of text and plan	executing tests, human review
Explanation	(printed descriptions)	generated from plan	generated from text

description, and much of the craft of using them lies in combining a functional request with a test-like check.

Where does the knowledge come from? Both historical systems are knowledge-starved by modern standards. The Heuristic Compiler's state description compiler knows exactly two operators; its functional description compiler can compile a routine only if a routine with the same process has already been compiled. KBEmacs knew "a few dozen" clichés; Waters estimated that a practical tool would need "many hundreds" [30, p. 55]. Both systems identified reuse of known solutions as the heart of the matter: Simon's functional compiler works by finding an analogous routine and adapting it, and the whole Programmer's Apprentice rests on the claim that most programs are built from standard pieces. What neither could solve was acquiring that knowledge at scale. Large language models can be seen as one answer to precisely this bottleneck: an implicit cliché library absorbed from an enormous body of code, at the price of having no explicit, inspectable representation of what a cliché is or what its roles are.

Who does what? The Heuristic Compiler was meant to be autonomous; KBEmacs was deliberately an apprentice, leaving design decisions to the programmer and taking over "the details." Waters's argument for the apprentice was practical: full automation was not near, and a semiexpert assistant could be useful at once. Modern tools have traversed the same axis in a few years, from completing a line, to writing a function on request, to agents that plan, edit, and test across a code base with varying degrees of supervision [6]. The question of where the programmer's decisions end and the machine's begin, central to the Programmer's Apprentice, is again a live design question.

How does the system know it is making progress? The Heuristic Compiler's executive is a loop: apply a compiler, test whether the routine has moved to a better kind of definition, and stop if it has not. Its state description compiler is difference reduction in the strict GPS sense. Modern coding agents commonly run a loop of a similar shape: make a change, run the tests or the compiler, examine the difference between what happened and what was wanted, and try again. The difference detector is now an executed test suite rather than a hand-coded comparison of states, and the operator is a learned model rather than a table of connections, but the architecture of means-end iteration is recognizable. KBEmacs, for its part, came with a self-test in which the expected output was code, which is exactly what made it possible to check the reanimation in 2026.

Representation and explanation. The two historical systems never treated code merely as text. The Heuristic Compiler builds IPL instructions out of description lists and prints them only at the end; KBEmacs reasons about plans and regenerates text after every command, taking pains to keep text and plan consistent when the user edits the text directly. That explicit representation is what allowed KBEmacs to describe a program in terms of its design ("The program MYSQRT is a successive-approximation . . . "). Today's assistants operate on text and also explain code fluently, but their explanations are generated from text, not read off an explicit representation of the design. The trade-off between the robustness of working with text, which KBEmacs found painfully expensive to analyze—Waters reports that "well over half of all the effort expended on KBEmacs went into implementing an analyzer and coder" [30, p. 55]—and the inspectability of working with plans, has not gone

away.

WHAT EXECUTABLE ARCHAEOLOGY CONTRIBUTES

As with the Logic Theorist [9], it is fair to ask what running these programs teaches that reading them and their documentation does not. Four answers stand out.

It reveals gaps between the published demonstration and the surviving program. The Heuristic Compiler's J3 example does not compile from the deck as listed; the published result requires a version of the data in which J3 lacks a functional description. This is not visible from the paper, nor from reading the executive, whose preference ordering looks perfectly sensible on the page. Similarly, the "functional descriptions" in Simon's demonstrations turn out to be hand-coded list structures; the English front end, which occupies a large part of the deck, is not exercised by any of the published examples.

It recovers semantics that survive only in the artifacts. The PDP-10 meaning of `boole`, the falsity of `(listp nil)`, and the integer nature of characters on the Lisp Machine are all documented somewhere, but nothing about KBEmacs's listings tells a reader that its correctness depends on them; the system simply does not work otherwise. On the IPL-V side, several J-functions had to be corrected in ways the Logic Theorist had never exercised, so each new program tests the interpreter as much as the interpreter tests the program.

It checks the transcription, and the record. Compensating parenthesis errors are invisible to every static check short of re-reading the scan; executing the code finds them. Conversely, the fact that a fresh transcription produces the Heuristic Compiler's published code character for character, or KBEmacs's 1984 output `equal` for `equal`, is strong evidence that the transcription and the interpreter are right along those paths, since errors would almost certainly have produced different code.

It makes historical claims checkable. Simon's 1963 claims about what the Heuristic Compiler did, and the 1984 self-test's claims about what KBEmacs did, can now be tested and extended. Most interestingly, KBEmacs's self-test records three *failures*. A reanimation that reproduces those failures, with the same wrong code, would be a sharper test of fidelity than one that reproduces only the successes.

RELATED WORK

This work belongs to a series of reanimations of early AI programs from their original sources. Lane et al. restored Weizenbaum's ELIZA to operation on a reconstructed CTSS running on an emulated IBM 7094 [7]. Shrager [21] and Moews and Shrager [9] reanimated two versions of the Logic Theorist, one in IPL-I and one in IPL-V; the IPL-V interpreter used for the Heuristic Compiler descends from that work.

The history of automatic programming has been surveyed by Balzer [2] and by Rich and Waters [17], the latter from the perspective of the Programmer's Apprentice. Deductive program synthesis, which formalizes the identification of proofs with programs that Simon made informally, begins with Green [5] and Manna and Waldinger [8]. Within the Programmer's Apprentice project, KBEmacs's capabilities and limits are assessed by Waters himself [30] and by Rich and Waters [16], [18]; Pitman describes its interface [13].

Reimplementing either system's ideas in a modern language would be a quite different undertaking from the present work. Both reanimations run the original code, transcribed from the surviving listings, on reconstructions of the abstract machine (IPL-V) or language environment (Zetalisp) it expected. For KBEmacs, a fuller reanimation would run the complete system, including its editor, on an emulated Lisp Machine; we have not attempted this.

NEXT STEPS

For the Heuristic Compiler, the largest piece of unexplored territory is the English front end: the scanner, dictionary, and parser (routines T50–T300 and U23–U99) and the routine that converts a parse into a functional description. Simon credited H. S. Kelly with "linguistic components for the compiler which are omitted from this report" [25, p. 506]; these routines may be them. Running them requires input sentences, which are not in the listing. The pencil corrections in the listing also remain to be evaluated.

For KBEmacs, the next step is to run the other five sequences of the 1984 self-test, aiming to reproduce both the three successes and the three failures; then the Ada front end, which would allow the `UNIT-REPAIR-REPORT` demonstration of Waters's papers to be rerun.

The transcriptions, interpreters, compatibility layer, and Hacker's Guides to both programs are open source [19], [20].

A PERSONAL CODA

Jeff Shrager

Both reanimations described here were carried out in collaboration with a modern AI coding assistant. It drafted the first transcriptions from the page images, wrote much of the KBEmacs compatibility layer and the IPL-V interpreter extensions, chased down the PDP-10 meaning of `boole`, and found the compensating parentheses. It also made mistakes, some of which took a while to find, and none of the results reported here were accepted until the program had reproduced its own historical output. In short, the division of labor was very much the one Waters proposed: I made the decisions about what to do and what counted as success, and the apprentice did much of the detailed work.

There is a pleasing circularity in this. Simon wondered whether programming could be treated as problem solving; Waters wondered whether a programmer could be given an apprentice that shared knowledge of standard algorithms. Forty and sixty-five years later, an apprentice of a kind neither of them imagined helped bring their programs back to life. Simon closed his paper by saying that the Heuristic Compiler was "an exploratory tool that is beginning to reveal to us something of the role played by these and other representations in the programming process" [25, p. 506]. Running it again, with help, suggests the exploration is not over.

ACKNOWLEDGMENTS

KBEmacs could not have been reanimated without Kent M. Pitman, who preserved the 1985 listings and made the scans available [14]. The Heuristic Coder listing was located in the Herbert A. Simon Papers at the Carnegie Mellon University Archives by Mia Golec and Emily Davis. David Moews's IPL-V Logic Theorist deck served as the regression test for the IPL-V interpreter. The ports of Spice Lisp Flavors and LetS used in the KBEmacs runtime are by g000001 [4]. RAND Memorandum RM-3588-PR was obtained from bitsavers. Listing excerpts from KBEmacs are copyright 1983/1984 Massachusetts Institute of Technology and are reproduced by permission under the notice in those listings.

Finally, we acknowledge Herbert Simon, H. S. Kelly, Richard Waters, Charles Rich, Kent Pitman, and the other members of the Programmer's Apprentice project, whose work this paper merely re-runs.

The transcriptions, the reanimation software, and portions of the drafting of this paper were produced with the aid of LLM-based tools, but all concepts and final editorial decisions were the author's alone.

BIBLIOGRAPHY

[1] J. W. Backus et al., "The FORTRAN automatic coding system," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 188–198.

[2] R. Balzer, "A 15 year perspective on automatic programming," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 11, pp. 1257–1268, Nov. 1985.

[3] M. Chen et al., "Evaluating large language models trained on code," Jul. 2021, *arXiv:2107.03374*.

[4] g000001, "spice-flavors" and "LetS," GitHub. Accessed: Oct. 10, 2026. [Online]. Available: https://github.com/g000001/spice-flavors, https://github.com/g000001/LetS

[5] C. Green, "Application of theorem proving to problem solving," in *Proc. 1st Int. Joint Conf. Artif. Intell.*, 1969, pp. 219–239.

[6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[7] R. Lane, A. Hay, A. Schwarz, D. M. Berry, and J. Shrager, "ELIZA reanimated: Restoring the mother of all chatbots to one of the world's first time-sharing systems," *IEEE Ann. Hist. Comput.*, vol. 47, no. 2, pp. 68–76, Apr.-Jun. 2025.

[8] Z. Manna and R. J. Waldinger, "Toward automatic program synthesis," *Commun. ACM*, vol. 14, no. 3, pp. 151–165, Mar. 1971.

[9] D. Moews and J. Shrager, "Executable archaeology: Reanimating the Logic Theorist," *IEEE Ann. Hist. Comput.*, submitted, 2026.

[10] A. Newell, J. C. Shaw, and H. A. Simon, "Report on a general problem-solving program," in *Proc. Int. Conf. Inform. Process.*, Paris: UNESCO, 1960, pp. 256–264.

[11] A. Newell, J. C. Shaw, and H. A. Simon, "Empirical explorations of the logic theory machine: A case study in heuristic," in *Proc. Western Joint Comput. Conf.*, 1957, pp. 218–230.

[12] A. Newell, F. M. Tonge, E. A. Feigenbaum, B. F. Green Jr., and G. H. Mealy, *Information Processing Language-V Manual*. Englewood Cliffs, NJ, USA: Prentice-Hall, Inc., 2nd ed., 1964.

[13] K. M. Pitman, "Interfacing to the Programmer's Apprentice," MIT AI Lab., Cambridge, MA, USA, Working Paper 244, Feb. 1983.

[14] K. M. Pitman, "netsettlement: kbe," GitHub (scans of *Knowledge-Based Editor*, four-volume listing, Jan. 1985). Accessed: Oct. 13, 2026. [Online]. Available:

https://github.com/netsettler/netsettlement/tree/main/kbe

[15] C. Rich and H. E. Shrobe, "Initial report on a LISP programmer's apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-354, Dec. 1976. (Also *IEEE Trans. Softw. Eng.*, vol. SE-4, no. 6, pp. 456–467, Nov. 1978.)

[16] C. Rich and R. C. Waters, "The Programmer's Apprentice project: A research overview," MIT AI Lab., Cambridge, MA, USA, AI Memo 1004, Nov. 1987.

[17] C. Rich and R. C. Waters, "Automatic programming: Myths and prospects," *Computer*, vol. 21, no. 8, pp. 40–51, Aug. 1988.

[18] C. Rich and R. C. Waters, *The Programmer's Apprentice*. New York, NY, USA: ACM Press, and Reading, MA, USA: Addison-Wesley, 1990.

[19] J. Shrager, "Executable Archaeology," GitHub (HeuristicCoder, IPL-V, and LogicTheorist directories). Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/ExecutableArchaeology

[20] J. Shrager, "kbemacs: MIT's KBEmacs from the January 1985 listings, running under SBCL," GitHub. Accessed: Oct. 16, 2026. [Online]. Available: https://github.com/jeffshrager/kbemacs

[21] J. Shrager, "Executable archaeology: Reanimating the Logic Theorist from its IPL-V source," Mar. 13, 2026, *arXiv:2603.13514v1*.

[22] J. Shrager. "An IPL-V interpreter in Common Lisp that runs the original Logic Theorist," GitHub (archived; superseded by [19]). [Online]. Available: https://github.com/jeffshrager/IPL-V

[23] H. A. Simon, "Experiments with a heuristic compiler," RAND Corp., Santa Monica, CA, USA, Paper P-2349, Jun. 30, 1961.

[24] H. A. Simon, "Heuristic Coder 7/16/61," IPL-V program listing, Herbert A. Simon Papers, Box 14, Folder FF965, Carnegie Mellon Univ. Archives, Pittsburgh, PA, USA.

[25] H. A. Simon, "Experiments with a heuristic compiler," *J. ACM*, vol. 10, no. 4, pp. 493–506, Oct. 1963.

[26] H. A. Simon, "The heuristic compiler," RAND Corp., Santa Monica, CA, USA, Memorandum RM-3588-PR, May 1963.

[27] R. C. Waters, "Automatic analysis of the logical structure of programs," Ph.D. dissertation, MIT, Cambridge, MA, USA, 1978. (MIT AI Lab. Tech. Rep. AI-TR-492.)

[28] R. C. Waters, "KBEmacs: A step toward the Programmer's Apprentice," MIT AI Lab., Cambridge, MA, USA, Tech. Rep. AI-TR-753, May 1985.

[29] R. C. Waters, "The Programmer's Apprentice: A session with KBEmacs," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 11, pp. 1296–1320, Nov. 1985.

[30] R. C. Waters, "KBEmacs: Where's the AI?" *AI Mag.*, vol. 7, no. 1, pp. 47–56, Spring 1986.

JEFF SHRAGER is with Bennu Climate, Inc., Palo Alto, CA 94301 USA. Contact them at jshrager@gmail.com.